



TÓPICOS EM INFORMÁTICA

Tratamento de entrada



ENTRADA

- As diferentes plataformas que a Libgdx suporta tem diferentes **tipos de entrada**.
 - Desktop e Browser (HTML5): O usuário se comunica com a aplicação por **mouse e teclado**.
 - Android e IOS: O movimento do mouse é substituído por uma **touch screen** e não existe teclado (pelo menos não usualmente). Além disso, dispositivos mobile contém **acelerômetro e até bússula** (não presente nos desktops).



ENTRADA

- A libgdx fornece meios de acessar esses diferentes tipos de input.
 - Cliques do mouse e toques na tela são considerados a mesma coisa. Ambos possuem uma função única na libgdx e funcionam igualmente nos dois casos.
 - Contudo, o mouse não tem multi-touch (mais de um dedo tocando a tela – nesse caso será reportado apenas um dedo ou um toque) e as touch screens não tem suporte a qual botão foi apertado (será considerado sempre como left click – botão esquerdo do mouse).
- Todas as funções de input podem ser acessadas a partir do módulo input (embora não seja a forma mais recomendada de fazer – veremos mais pra frente):

```
boolean isPressed = Gdx.input.isKeyPressed(Keys.A);
```



DESATIVANDO ENTRADAS

- Em alguns casos você pode acabar não precisando de **todas as formas de input disponíveis** pela libgdx como, por exemplo, a bússula e o acelerômetro. Nesses casos é ideal que sejam desativados para evitar consumo de bateria no Android.

```
public class MyGameActivity extends AndroidApplication {  
    @Override  
    public void onCreate (Bundle savedInstanceState) {  
        super.onCreate(savedInstanceState);  
        AndroidApplicationConfiguration config = new AndroidApplicationConfiguration();  
        config.useAccelerometer = false;  
        config.useCompass = false;  
        initialize(new MyGame(), config);  
    }  
}
```



ATIVANDO ENTRADAS

- O **giroscópio** vem desativado por padrão para reduzir o consumo de bateria mas, se necessário, é possível ativá-lo:

```
public class MyGameActivity extends AndroidApplication {  
    @Override  
    public void onCreate (Bundle savedInstanceState) {  
        super.onCreate(savedInstanceState);  
        AndroidApplicationConfiguration config = new AndroidApplicationConfiguration();  
        config.useGyroscope = true;  
        initialize(new MyGame(), config);  
    }  
}
```



CHECANDO ENTRADAS DISPONÍVEIS

- É possível checar as entradas disponíveis na sua aplicação da seguinte forma:

```
boolean hardwareKeyboard = Gdx.input.isPeripheralAvailable(Peripheral.HardwareKeyboard);  
boolean multiTouch = Gdx.input.isPeripheralAvailable(Peripheral.MultitouchScreen);
```

- É importante notar que caso o teclado do android não esteja **aberto e funcionando** no momento da chamada da função, ela retornará false.



INPUT POLLING

- As formas de fazer o tratamento de input, envolvem:

```
boolean isAPressed = Gdx.input.isKeyPressed(Keys.A);
```

```
boolean isTouched = Gdx.input.isTouched();
```

*// Retornará verdadeiro apenas se a tela estiver sendo tocada no
Momento exato da chamada da função/método isTouched*

```
boolean firstFingerTouching = Gdx.input.isTouched(0);
```

```
boolean secondFingerTouching = Gdx.input.isTouched(1);
```

```
boolean thirdFingerTouching = Gdx.input.isTouched(2);
```

- O primeiro dedo a encostar corresponde ao índice 0, o segundo corresponde ao índice 1, e assim vai.



LÓGICA DOS TOQUES NA TELA

- Primeiro dedo encosta na tela → 0
- Segundo dedo encosta na tela → 1
- Terceiro dedo encosta na tela → 2

- Segundo dedo levanta → índice 1 é liberado
- Primeiro dedo levanta → índice 0 é liberado, nesse ponto apenas o 2 está sendo usado
- Um outro dedo encosta na tela → 0, por ser o menor índice livre disponível



COORDENADAS DO TOQUE

- É possível pegar as coordenadas do toque da seguinte forma:

```
int firstX = Gdx.input.getX();  
int firstY = Gdx.input.getY();  
int secondX = Gdx.input.getX(1);  
int secondY = Gdx.input.getY(1);
```

- O índice passado como parâmetro **indica o índice do dedo/toque**.
- É possível pegar a pressão do toque com o seguinte método (retornará um valor entre 0 e 1):

`Gdx.input.getPressure()`



BOTÃO DO MOUSE

- No desktop é possível também checar que botão está sendo pressionado:

```
boolean leftPressed = Gdx.input.isButtonPressed(Input.Buttons.LEFT);  
boolean rightPressed = Gdx.input.isButtonPressed(Input.Buttons.RIGHT);
```

- No android qualquer evento de toque será **emulado como um toque do botão esquerdo do mouse** (equivalente).



INPUT PROCESSOR

- Embora seja possível ler e tratar entradas com o seguinte código:

```
if (Gdx.input.isTouched()) {  
    System.out.println("Aconteceu um clique em  
x=" + Gdx.input.getX() +  
    ", y=" + Gdx.input.getY());  
}
```

- O mais indicado é utilizar a interface `InputProcessor`, disponível na libgdx. O `InputProcessor` funciona como o padrão observador (observer). Primeiro implementamos um listener e então adicionamos ele à libgdx.



INPUT PROCESSOR

- Crie uma classe que **implemente o InputProcessor**.
- Depois disso, adicione o seguinte código dentro do código da sua aplicação, isso irá ativar a chamada do seu input processor caso um dos eventos aconteça:

```
Gdx.input.setInputProcessor(meuInputProcessor);
```



ENTRADAS REALMENTE NECESSÁRIAS

- É possível construir **toda a sua aplicação utilizando apenas os eventos:**
 - **touchDown:** quando um dedo encostou na tela (ou botão do mouse pressionado)
 - **touchUp:** quando um dedo saiu da tela (ou botão do mouse soltado)
 - **keyDown:** tecla é apertada
 - **keyUp:** tecla é levantada (solta)
 - **keyTyped:** um unicode character foi gerado pelo keyboard
- Embora existam outros **eventos também interessantes**, é possível **ignorá-los** em função de uma aplicação adaptada a todos os dispositivos:
 - **scrolled:** rolou o scroll do mouse
 - **touchDragged:** quando o dedo ou o toque do mouse foi arrastado pela tela
 - **mouseMoved:** é ativado sempre que o mouse saiu do lugar (apenas desktop)



EXERCÍCIO

- Crie uma classe principal da sua aplicação implementar um input processor, **depois chame o seguinte comando para fazê-lo funcionar**: `Gdx.input.setInputProcessor(meuParamInputProcessor);`



INPUT ADAPTER

- O **InputAdapter** implementa todos os métodos do **InputProcessor** retornando false para cada método. Você pode estender o InputAdapter e implementar apenas os métodos que você precisa:

```
Gdx.input.setInputProcessor(new InputAdapter () {  
    @Override  
    public boolean touchDown (int x, int y, int pointer, int button) {  
        // código quando o dedo encostar na tela / clicar na tela  
        return true; // retorna verdadeiro pra indicar que foi tratado  
    }  
  
    @Override  
    public boolean touchUp (int x, int y, int pointer, int button) {  
        // quando soltar o dedo  
        return true; // retorna verdadeiro pra indicar que foi tratado  
    }  
});
```



INPUT MULTIPLEXER

- Por fim, pode ser interessante ter **vários InputProcessors** dentro da sua aplicação, por exemplo:
 - Um InputProcessor pra tratar as entradas de **clique no mapa**.
 - Outro InputProcessor para tratar a interação com os menus (botões, sliders, textboxes, etc).
- Isso é obtido através do **InputMultiplexer**:

```
InputMultiplexer multiplexer = new InputMultiplexer();  
multiplexer.addProcessor(new MyUiInputProcessor());  
multiplexer.addProcessor(new MyGameInputProcessor());  
Gdx.input.setInputProcessor(multiplexer);
```




EXERCÍCIO

- Rode uma aplicação com **pelo menos um exemplo de `InputProcessor` e `InputMultiplexer`**, testando a sua funcionalidade (se está tudo ok).

CONTROLE

- A libgdx também fornece um conjunto de funções para **trabalhar com controles** (de Xbox, por exemplo):





CONTROLE

- Inicialmente, é possível buscar os **controles conectados** com o seguinte código:

```
for (Controller controller : Controllers.getControllers()) {  
    Gdx.app.log(TAG, controller.getName());  
}
```

- Esse código respeita, quase sempre, a **ordem em que os controles foram conectados ao PC**.
 - Uma vez com acesso ao *controller*, é possível fazer:

```
boolean buttonPressed = controller.getButton(buttonCode);  
float axisValue = controller.getAxis(axisCode);  
...
```

- Isso retornará o estado do botão (**no momento da chamada**).



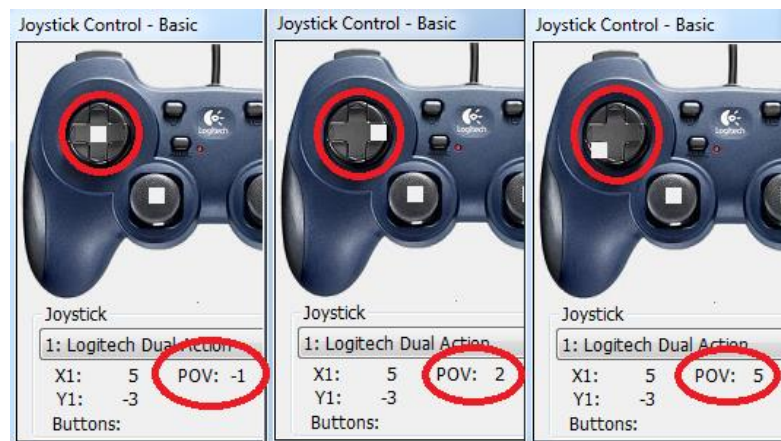
CONTROLE

- O **código do botão** deve ser adivinhado por tentativa e erro (**não existe uma padronização de código** nos diferentes controles do mercado).
 - O mais interessante é **deixar o usuário configurar (mapear) seu próprio controle**. Escutar os seus inputs e associá-los aos comandos corretos.
- Assim como no outro caso (do **InputProcessor**), existe também o **ControllerListener**.

CONTROLE

- Os eventos para o controle são:

```
public interface ControllerListener {  
    public void connected(Controller controller);  
    public void disconnected(Controller controller);  
    public boolean buttonDown (Controller controller, int buttonCode);  
    public boolean buttonUp (Controller controller, int buttonCode);  
    public boolean axisMoved (Controller controller, int axisCode, float value);  
    public boolean povMoved (Controller controller, int povCode, PovDirection value);  
    public boolean xSliderMoved (Controller controller, int sliderCode, boolean value);  
    public boolean ySliderMoved (Controller controller, int sliderCode, boolean value);  
    public boolean accelerometerMoved (Controller controller, int accelerometerCode, Vector3 value);  
}
```





CONTROLE

- Para adicionar o listener na libgdx:

```
Controllers.addListener(listener); // receberá evento de todos os controles  
controller.addListener(listener); // receberá evento apenas do controller mencionado
```

- É possível também estender a classe ControllerAdapter:

```
controller.addListener(new ControllerAdapter() {  
    @Override  
    public void connected(Controller controller) {  
        ...  
    }  
    public void disconnected(Controller controller) {  
        ...  
    }  
});
```



GESTOS

- A Libgdx fornece um detector de gestos em que você pode reconhecer:
 - **longPress**: toques longos.
 - **tap**: toques rápidos.
 - **pan**: o usuário passa com o dedo pela tela.
 - **panStop**: quando para de fazer o pan.
 - **fling**: arrastar – jogar pro lado, pra cima, pra baixo, etc. Famoso **swipe**.
 - **zoom**: o usuário coloca dois dedos na tela e faz o movimento de zoom (separando os dois dedos).
 - **pinch**: similar ao zoom, porém mais indicado para implementar rotações indicadas pelos dois dedos, por exemplo.



GESTOS

- De forma similar aos anteriores, é preciso implementar a interface `GestureListener`, passá-la para o `GestureDetector` e adicionar essa instância ao `InputProcessor`:

```
Gdx.input.setInputProcessor(new GestureDetector(new MyGestureListener()));
```




TEXT INPUT LISTENERS

- É possível escutar por entradas de texto de forma bem fácil na Libgdx:

```
public class MyTextInputListener implements TextInputListener {  
    @Override  
    public void input (String text) {  
    }  
    @Override  
    public void canceled () {  
    }  
}
```

- O método input será chamado quando o usuário **entrar com um texto/string**. O método canceled será chamado se o usuário **fechar o diálogo ou apertar o botão de voltar no Android**. Para chamar o diálogo utilize:

```
MyTextInputListener listener = new MyTextInputListener();  
Gdx.input.getTextInput(listener, "Título", "Valor inicial do textfield", "Dica de preenchimento");
```



OUTRAS ENTRADAS

- Vamos propositadamente **pular os seguintes Inputs** e a forma de tratá-los:
 - Acelerômetro (<https://github.com/libgdx/libgdx/wiki/Accelerometer>)
 - Bússula (<https://github.com/libgdx/libgdx/wiki/Compass>)
 - Vibrador (<https://github.com/libgdx/libgdx/wiki/Vibrator>)
- Ou ainda, forçar o posicionamento do cursor no centro (**em jogos de tiro** – por exemplo):
 - <https://libgdx.com/wiki/input/cursor-visibility-and-catching>
- Mostrar uma janela de confirmação caso o usuário de android **aperte o botão voltar** (pra confirmar a ação):
 - <https://libgdx.com/wiki/input/back-and-menu-key-catching>



CHAMAR TECLADO VISUAL

- Para chamar o teclado do Android e IOS, a seguinte função ou método pode ser utilizada:

```
Gdx.input.setOnscreenKeyboardVisible(true);
```

- Uma vez que o teclado esteja visível, **qualquer clique nas teclas será reportado e tratado pelos eventos vistos anteriormente**:
 - keyUp
 - keyDown
 - keyTyped



EXERCÍCIO

- Faça uma aplicação em que você consiga se comunicar com a utilização do teclado e esse **texto escrito por você chegue de alguma forma na aplicação** (pode ser printado no log ou console ao final).